



DOCUMENTO TÉCNICO

Arquitetura do ResolveAi

Aplicação de gestão de ocorrências com frontend
React, API Express, PostgreSQL no Neon e controle
administrativo de acesso.

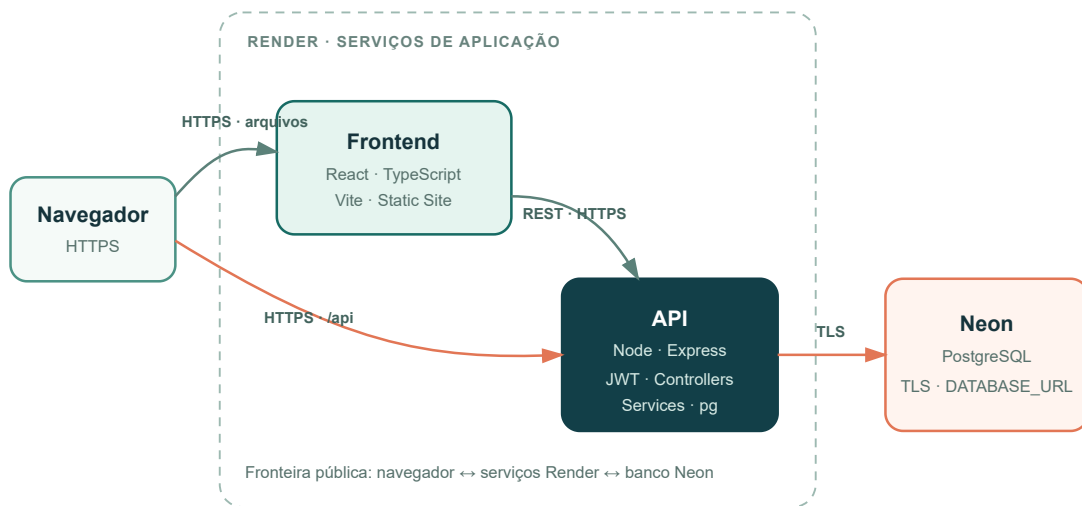
Frontend: React + Vite Backend: Node.js + Express Dados: PostgreSQL

Visão baseada na configuração do repositório e nos alvos Docker documentados.

RESOLVEAI / ARQUITETURA

Arquitetura publicada

No ambiente publicado, frontend e API são serviços separados no Render. O navegador acessa ambos por HTTPS. A API é a única camada que conecta ao banco PostgreSQL gerenciado no Neon.



- Frontend estático servido pelo Render.
- API autentica, autoriza e aplica regras de negócio.
- Neon é serviço gerenciado externo; credenciais ficam em segredo de ambiente.
- Chamadas HTTPS/TLS atravessam fronteira de rede.

Importante: em produção no Render, o navegador chama diretamente a URL pública da API. O proxy Nginx pertence ao Compose local e não faz parte deste desenho.

Docker Compose local

As imagens de produção têm dois alvos. O container Nginx serve o bundle compilado do React e encaminha chamadas da API pela rede privada do Compose. O container Node acessa o Neon externo usando variáveis de ambiente.

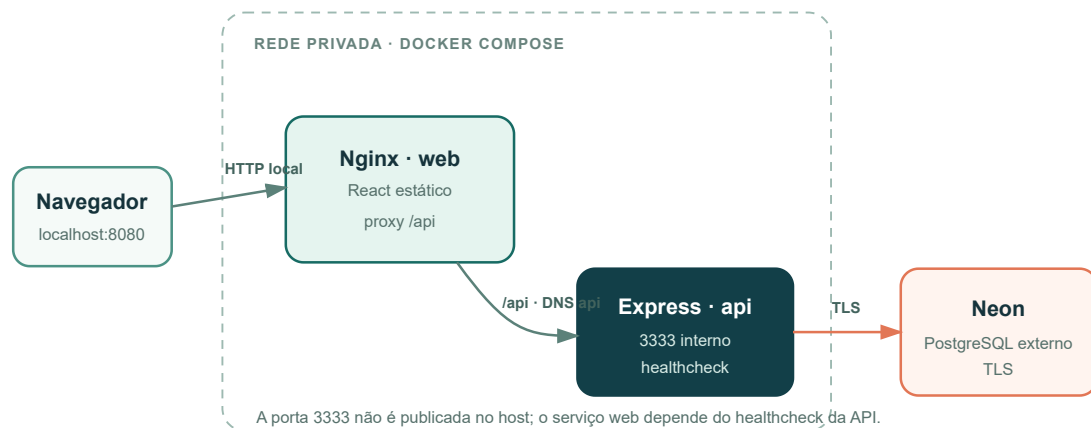


Imagem API

Node 22, dependências de runtime e código TypeScript compilado. Testes executam no estágio de build.

Imagem web

Bundle Vite compilado em build stage e servido por Nginx Alpine.

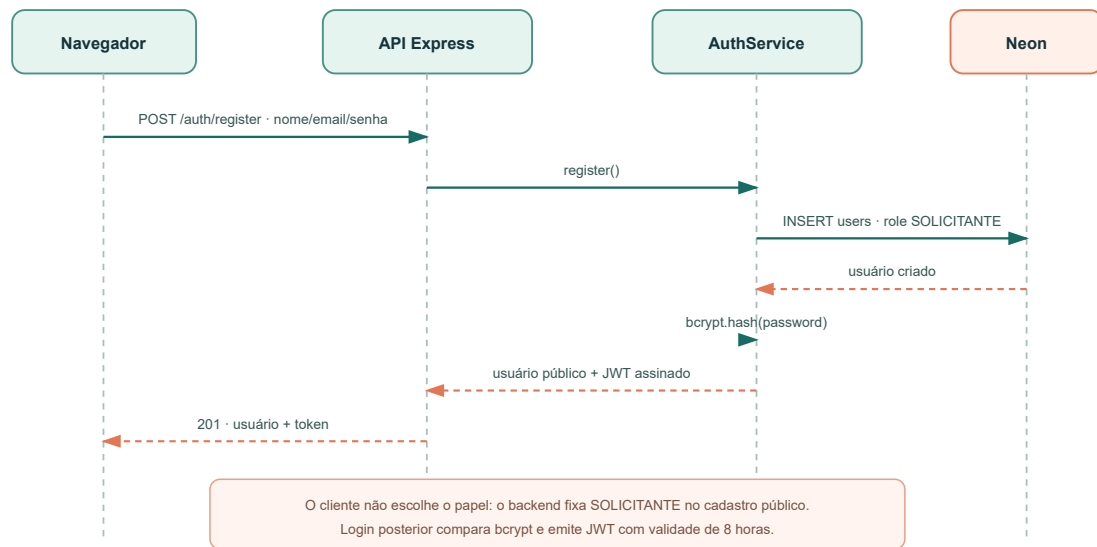
Persistência

Não existe banco PostgreSQL no Compose. O banco continua no Neon; não há volume local de dados.

Limite de confiança: tráfego do navegador termina no Nginx local. Nginx → API usa rede interna do Compose; API → Neon sai pela rede usando TLS.

Cadastro e autenticação

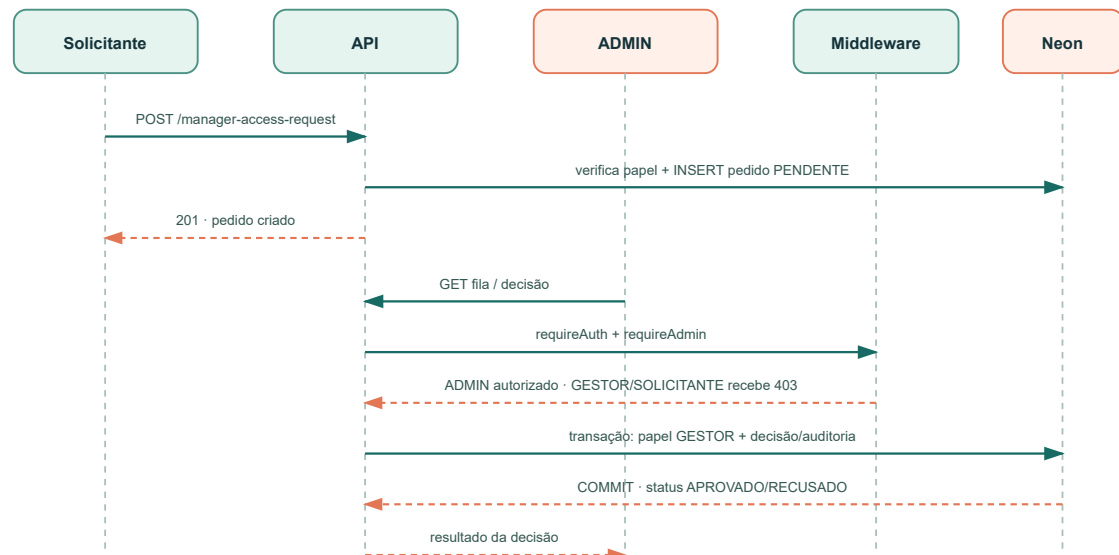
O papel enviado pelo cliente não é usado para promover uma conta. O serviço cria registros públicos como SOLICITANTE; login valida a senha e emite um JWT.



Credenciais: a senha armazenada é hash bcrypt. O JWT é retornado ao cliente e usado como Bearer nas rotas protegidas.

Pedido e aprovação de gestor

O fluxo de gestão tem duas barreiras: o solicitante autenticado abre o pedido; apenas um usuário ADMIN autenticado pode consultar a fila e decidir. A promoção para GESTOR ocorre dentro da mesma transação que registra a decisão.



Dados relevantes

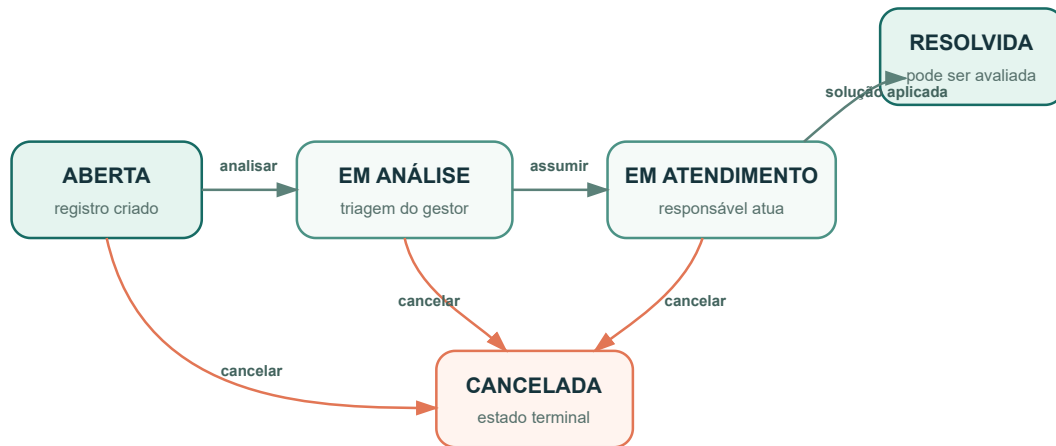
manager_access_requests guarda solicitante, justificativa, status, decisão, administrador e datas. Um índice parcial impede pedidos pendentes duplicados por usuário.

Bootstrap do primeiro ADMIN

A conta inicial é promovida manualmente no banco por uma operação administrativa controlada. Não existe rota pública para criar ADMIN.

Estados da ocorrência

O fluxo abaixo corresponde às transições aceitas pela camada de serviço. Resolvida e Cancelada são estados terminais; cada transição exige observação e registra ator/data no histórico.

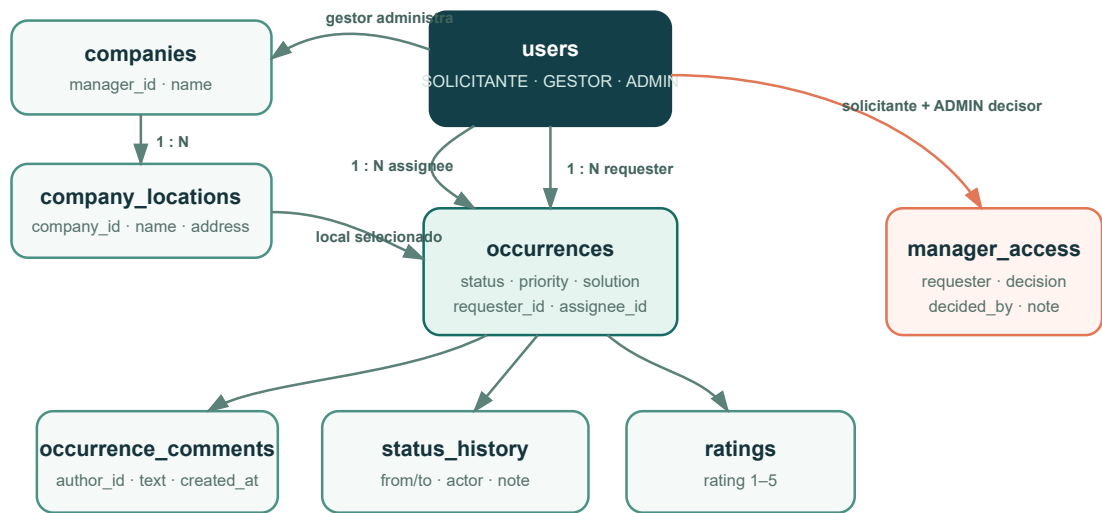


Toda mudança exige observação; histórico persiste status anterior/novo, data e usuário responsável.

Regras: ABERTA → EM ANÁLISE → EM ATENDIMENTO → RESOLVIDA. O cancelamento pode ocorrer antes da conclusão. Estados terminais não reabrem pelo fluxo atual.

Modelo de domínio

Gestores cadastram empresas e endereços curtos; solicitantes escolhem a empresa e o prédio ao abrir uma ocorrência. A chave composta garante que o endereço pertence à empresa. As entidades preservam histórico, atribuição e avaliação.



A view `occurrence_summary` agrega ocorrência, nomes de requester/assignee e rating.

Controles e limites

Área	Controle implementado	Observação operacional
Cadastro	Papel público fixado em SOLICITANTE.	Não confiar em role recebido do navegador.
Rotas	JWT; middleware <code>requireManager</code> e <code>requireAdmin</code> .	Autorização verificada no backend.
Empresas	Gestores criam empresas e endereços; lista, detalhe, atribuição e alterações são limitadas às empresas administradas.	ADMIN possui visão global. Ocorrências antigas sem vínculo exigem classificação administrativa.
Senhas	Hash <code>bcrypt</code> antes de persistir.	Segredos de ambiente não devem entrar no repositório.
Imagens	PNG/JPEG/WebP até 1 MB são enviados como data URL.	O MVP grava o conteúdo em <code>image_url</code> TEXT no Neon; migrar para object storage antes de maior escala.
Banco	Parâmetros SQL preparados; conexão via <code>DATABASE_URL</code> ; imagem, comentário, rating e histórico persistidos.	Neon é externo; não há banco em container local.
CORS/JWT	CORS atualmente permite origem ampla; código possui fallback de segredo de desenvolvimento.	Para produção, restrinja CORS e defina <code>JWT_SECRET</code> forte no Render.

Documento gerado a partir dos arquivos de aplicação, schema PostgreSQL, Dockerfile, Compose e configuração Nginx. Serviços hospedados e variáveis reais foram omitidos intencionalmente.